Room - OWASP Top 10 2025: Application Design Flaws

This room breaks each 4 of the OWASP Top 10 2025 categories. In this room, you will learn about the categories that are related to failures in architecture and system design. You will put the theory into practice by completing supporting challenges. The following categories are covered in this room:

1. AS02: Security Misconfigurations
2. AS03: Software Supply Chain Failures
3. AS04: Cryptographic Failures
4. AS06: Insecure Design

Security Misconfigurations occur when systems, applications, servers, or cloud services are set up insecurely. These are usually deployment or configuration mistakes rather than coding bugs.

Why it matters:

- Misconfigurations can expose sensitive data, allow unauthorized access, or help attackers gain control of systems.
- Even a small mistake, like a public cloud bucket or default password, can lead to a major breach.

Example:

- In 2017, Uber exposed sensitive user data because an AWS S3 backup bucket was publicly accessible.

Common security misconfigurations:

- Default usernames/passwords left unchanged
- Unnecessary services exposed to the internet
- Misconfigured cloud storage or permissions
- APIs without proper authentication/authorization
- Detailed error messages revealing system information
- Outdated software with known vulnerabilities
- Exposed AI/ML endpoints without access control

How to prevent them:

- Harden default settings and remove unused services
- Use strong authentication and least privilege

- Restrict network exposure and segment systems
- Regularly patch and update software
- Hide stack traces and sensitive error details
- Audit cloud permissions frequently
- Protect AI endpoints with authentication and monitoring
- Add automated security/configuration checks into deployment pipelines

Core lesson:

- Many breaches happen not because attackers are highly advanced, but because systems are configured insecurely. Proper setup, monitoring, and maintenance are critical parts of cybersecurity.

Software Supply Chain Failures happen when applications depend on compromised, outdated, or untrusted third-party software, libraries, services, or AI models. The weakness is not in your own code, but in the components you rely on.

Why it matters:

- Modern software heavily depends on external packages, APIs, cloud services, and AI models.
- If one dependency is compromised, attackers may gain access to thousands of systems at once.
- These attacks are difficult to detect because the malicious component often appears trusted.

Example:

- In 2021, SolarWinds suffered a major supply chain attack where attackers inserted malicious code into trusted Orion software updates, affecting many organizations worldwide.

In AI systems:

- Unverified AI models or datasets may contain hidden backdoors, malicious behaviors, or biased outputs that can leak data or compromise systems.

Common patterns:

- Using unverified or outdated dependencies
- Automatically installing updates without verification
- Insecure CI/CD or build pipelines
- Poor tracking of software origin and licenses
- Lack of vulnerability monitoring after deployment
- Blind trust in third-party AI models

How to protect against supply chain failures:

- Verify and audit all third-party software and AI models
- Regularly monitor and patch dependencies
- Sign and verify software updates/packages
- Secure CI/CD pipelines against tampering
- Track provenance and licensing of dependencies
- Monitor runtime behavior for suspicious activity
- Include supply chain security in the SDLC (Software Development Life Cycle)

Core lesson:

- Your system is only as secure as the weakest dependency in your software supply chain. Trusting third-party components without verification can expose the entire application to compromise.

Cryptographic Failures occur when encryption or secret management is implemented incorrectly or missing entirely. These weaknesses can expose sensitive data such as passwords, tokens, encryption keys, or personal information.

Why it matters:

- Cryptography protects data in transit and at rest.
- If encryption is weak or secrets are exposed, attackers can steal or modify sensitive information.
- Failures may lead to account compromise, data leaks, or complete system breaches.

Common cryptographic mistakes:

- Using weak/deprecated algorithms like MD5, SHA-1, or ECB mode
- Hard-coded passwords, API keys, or encryption keys
- Poor key management or lack of key rotation
- Sensitive data stored or transmitted without encryption
- Invalid or self-signed TLS certificates
- AI/ML systems exposing secrets or sensitive model data

How attackers exploit these issues:

- Man-in-the-middle (MITM) attacks
- Brute-forcing weak encryption keys
- Extracting exposed secrets from source code or configuration files
- Intercepting unencrypted network traffic

How to prevent cryptographic failures:

- Use strong modern encryption such as AES-GCM or ChaCha20-Poly1305
- Enforce secure TLS configurations (TLS 1.3)
- Store secrets in secure vaults like:
    - AWS KMS
    - Microsoft Azure Key Vault

    - HashiCorp Vault
- Rotate keys and secrets regularly
- Maintain proper certificate and key inventories
- Never expose secrets in AI systems or automation tools

Challenge summary:

- The application at `10.48.173.40:5004` contains a cryptographic weakness.
- Your goal is to inspect the application, locate the exposed/decryptable key, and use it to decrypt the protected file.

Core lesson:

- Encryption is only effective when implemented and managed securely. Weak algorithms, exposed keys, or poor secret handling can completely undermine security.

Insecure Design happens when security flaws are built into the system's architecture, workflows, or business logic from the beginning. These are design-level problems, not simple coding mistakes.

Why it matters:

- Insecure designs cannot usually be fixed with small patches.
- The core logic, trust boundaries, or assumptions themselves are unsafe.
- AI systems increase the risk because developers may trust model outputs too much or give AI excessive authority.

Example:

- Clubhouse initially assumed users would only access the platform through the mobile app. However, backend APIs lacked proper authentication, allowing researchers to directly access sensitive user and conversation data.

Common insecure design patterns:

- Weak business logic (password recovery, approval flows)
- Incorrect assumptions about users or AI behaviour
- AI systems with excessive permissions
- Missing safeguards for LLMs and automation agents
- Debug/test bypasses left in production
- Lack of threat modelling and abuse-case analysis

AI-related insecure design risks:

- Prompt injection attacks
- Blind trust in AI-generated outputs
- Poisoned or backdoored AI models

- AI agents performing actions without validation
- Sensitive data leakage through prompts

How to design securely:

- Treat AI models as untrusted until validated
- Separate system prompts from user input
- Validate all model inputs and outputs
- Require human review for high-risk AI actions
- Protect sensitive data and minimize prompt exposure
- Apply least privilege to users, APIs, and AI agents
- Build authentication and authorization correctly
- Continuously perform threat modelling and abuse testing
- Verify dependencies and third-party components regularly

Core lesson:

- Security must be built into the design phase, not added later. Poor assumptions, flawed workflows, or over-trusting AI can create vulnerabilities that affect the entire system.